Manual de Introdução a CC

Bruno Silva

O manual do iniciante em C: aprenda o básico sobre a linguagem de programação C em apenas algumas horas

Figure 1: O manual do iniciante em C: aprenda o básico sobre a linguagem de programação C em apenas algumas horas

Artigo original: The C Beginner's Handbook: Learn C Programming Language basics in just a few hours

O manual do iniciante em C segue a regra dos 80/20. Você aprenderá 80% da linguagem de programação C em 20% do tempo.

Essa abordagem dará a você uma visão geral bem engendrada da linguagem.

Este manual não busca cobrir tudo o que existe em relação ao C. Ele se concentra no básico da linguagem, tentando simplificar os tópicos mais complexos.

Sumário

1. Introdução ao C
2. Variáveis e tipos
3. Constantes
4. Operadores
5. Condicionais
6. Laços
7. Arrays
8. Strings
9. Ponteiros
10. Funções
11. Entrada e saída
12. Escopo das variáveis
13. Variáveis estáticas
14. Variáveis globais
15. Definições de tipo
16. Tipos enumerados
17. Estruturas
18. Parâmetros da linha de comando
19. Arquivos de cabeçalho (*headers*)
20. O pré-processador

Introdução ao C

Voltar ao sumário.

O C é, provavelmente, a linguagem de programação mais amplamente conhecida. Ela é usada como linguagem de referência para os cursos de ciência da computação em todo mundo, sendo, possivelmente, a linguagem que as pessoas mais aprendem nas escolas, juntamente com Python e Java.

Eu me lembro de ela ser minha segundo linguagem de programação na vida, depois de aprender Pascal.

O C não é apenas o que os estudantes usam para aprender programação. Ela não é uma linguagem acadêmica. Eu diria, até, que não é a linguagem mais fácil, pois ela é uma linguagem de programação de nível mais baixo.

Hoje, o C é amplamente usado em dispositivos integrados, alimentando a maioria dos servidores de Internet, os quais são criados usando o Linux. O kernel do Linux foi criado em C, o que significa que o C também está no núcleo de todos os dispositivos Android. Podemos dizer que código em C está em execução em uma boa parte do mundo todo. Neste exato momento. Isso é algo bastante notável.

Quando foi criado, o C era considerado uma linguagem de alto nível, pois era portável entre as diversas máquinas. Hoje, consideramos algo óbvio que um programa seja escrito no Mac, no Windows ou no Linux, talvez usando o Node.js ou o Python.

Houve um tempo, no entanto, em que esse, definitivamente, não era o caso. O que o C ofereceu na época foi o fato de a linguagem ser simples de implementar, tendo um compilador que poderia ser portado facilmente para máquinas diferentes.

Eu falei em compilador: o C é uma linguagem de programação compilada, como o Go, o Java, o Swift ou o Rust. Outras linguagens de programação populares, como o Python, o Ruby ou o JavaScript, são interpretadas. A diferença é consistente: uma linguagem compilada gera um arquivo binário que pode ser executado e distribuído diretamente.

O C não tem um coletor de lixo. Isso quer dizer que precisamos lidar com o gerenciamento da memória por nossa conta. Essa é uma tarefa complexa e exige muita atenção para evitar bugs, mas também é o que faz com que o C seja ideal para escrever programas para dispositivos integrados como o Arduino.

O C não esconde por debaixo dos panos a complexidade e as habilidades da máquina. Você tem muito poder quando sabe o que fazer com ele.

Deixe-me apresentar o primeiro programa em C agora, o qual chamaremos de “Hello, World!” (“Olá, mundo!” em inglês)

hello.c

```c
#include <stdio.h>

int main(void) {
    printf("Hello, World!");
}
```

Vamos descrever o código-fonte do programa: primeiro, importaremos a biblioteca `stdio` (o nome significa "biblioteca de entrada e saída padrão" - em inglês, *STanDard Input and Output*).

Essa biblioteca nos dá acesso às funções de entrada e saída.

O C é uma linguagem muito pequena em seu núcleo. Tudo o que não é parte do núcleo é fornecido por meio de bibliotecas. Algumas dessas bibliotecas foram criadas por programadores "normais" e disponibilizadas para que outros as utilizassem. Algumas outras bibliotecas foram criadas dentro do compilador, como a `stdio`, entre outras.

A `stdio` é a biblioteca que fornece a função `printf()`.

A função está envolvida em uma função `main()`. A função `main()` é o ponto de entrada de qualquer programa em C.

Mas, enfim, o que é uma função?

Uma função é uma rotina que recebe um ou mais argumentos e retorna um único valor.

No caso de `main()`, a função não recebe argumentos e retorna um número inteiro. Identificamos isso usando a palavra-chave `void` como argumento e a palavra-chave `int` para o valor de retorno.

A função tem um corpo, que é envolvido por chaves. Dentro do corpo da função, temos todo o código de que a função necessita para realizar suas operações.

A função `printf()` está escrita de modo diferente, como é possível ver. Ela não tem um valor de retorno definido e passamos a ela uma string, envolvida por aspas duplas. Não especificamos o tipo de argumento.

Isso ocorre porque essa é uma invocação de função. Em algum lugar, dentro da biblioteca `stdio`, `printf` está definida assim:

```c
int printf(const char *format, ...);
```

Você não precisa entender o que isso significa agora, mas, resumindo, essa é a definição. Quando chamamos `printf("Hello, World!");`, a função será executada.

A função `main()` que definimos acima:

```c
#include <stdio.h>

int main(void) {
    printf("Hello, World!");
}
```

será executada pelo sistema operacional quando o programa for executado.

Como executamos um programa em C?

Conforme dissemos acima, C é uma linguagem compilada. Para executar o programa, primeiro precisamos compilá-lo. Todo computador Linux ou macOS já vem com um compilador de C integrado. No caso do Windows, você pode usar o Windows Subsystem for Linux (WSL).

Seja como for, ao abrir a janela do terminal, você pode digitar `gcc`. Esse comando retornará uma mensagem de erro, dizendo que você não especificou nenhum arquivo:

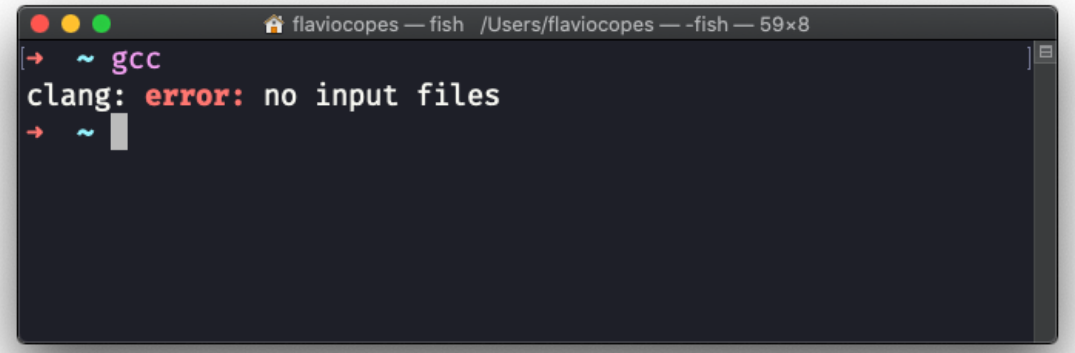

Figure 2: Screen-Shot-2020-01-29-at-10.10.50

Isso é bom. Significa que o compilador de C está lá e que podemos começar a utilizá-lo.

Agora, digite o programa acima em um arquivo `hello.c`. Você pode usar qualquer editor, mas, para fins de simplificar a vida, usarei o editor `nano` na linha de comando:

Digite o programa:

Agora, pressione `ctrl-X` para sair do nano:

Confirme pressionando a tecla `y` , e pressione Enter para confirmar o nome do arquivo:

Isso, agora, nos levará de volta ao terminal:

Em seguida, digite:

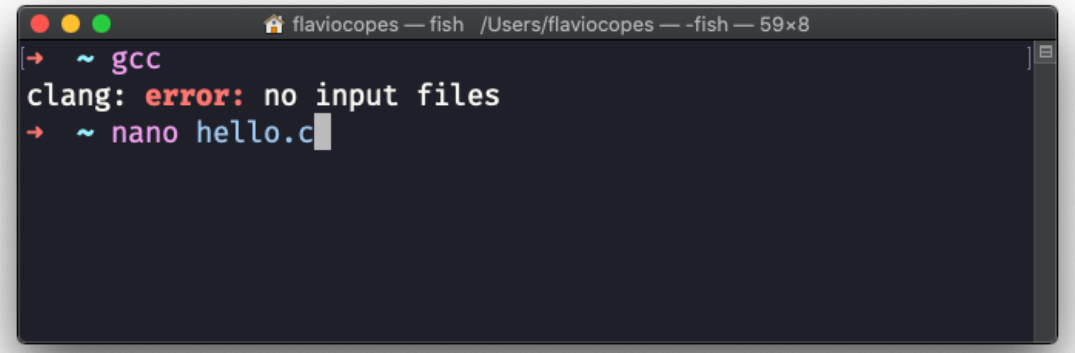

Figure 3: Screen-Shot-2020-01-29-at-10.11.39

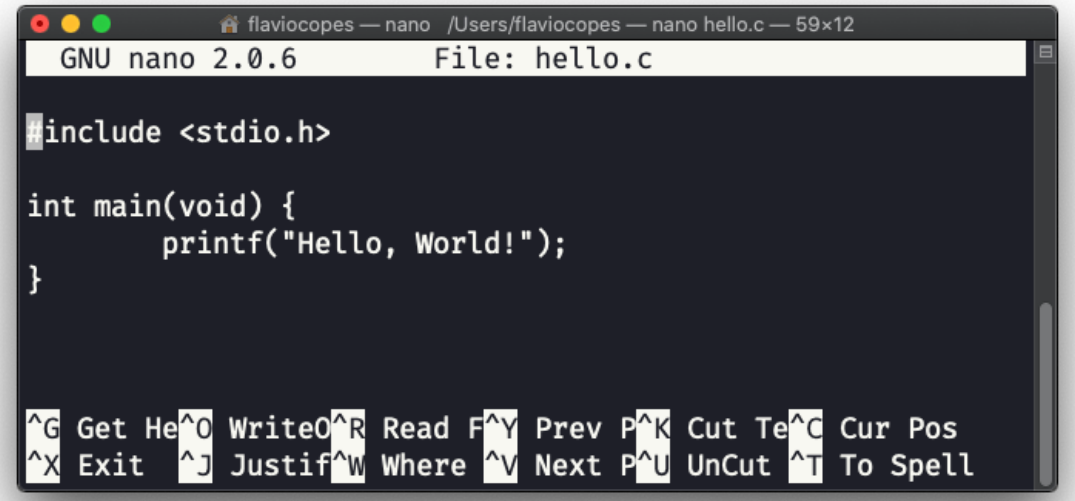

Figure 4: Screen-Shot-2020-01-29-at-10.16.52

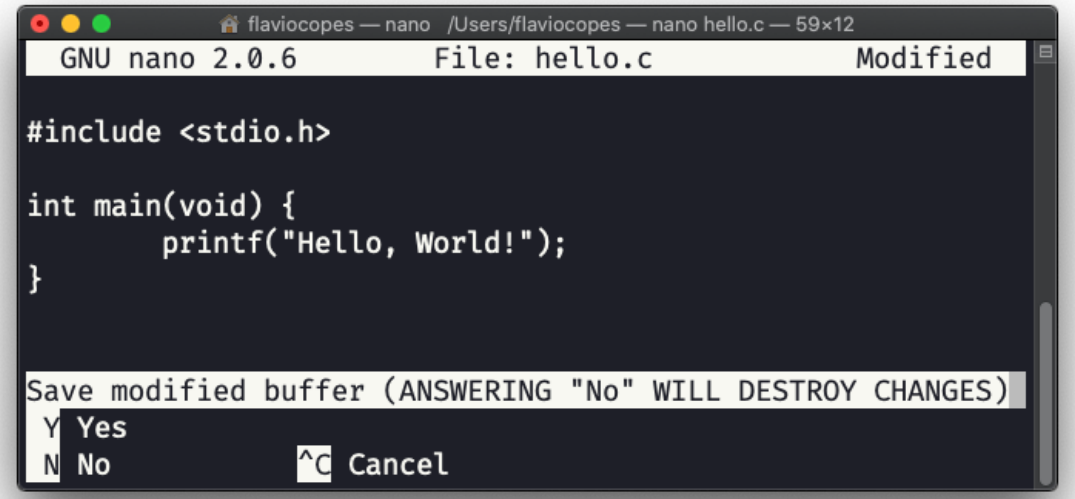

Figure 5: Screen-Shot-2020-01-29-at-10.18.11

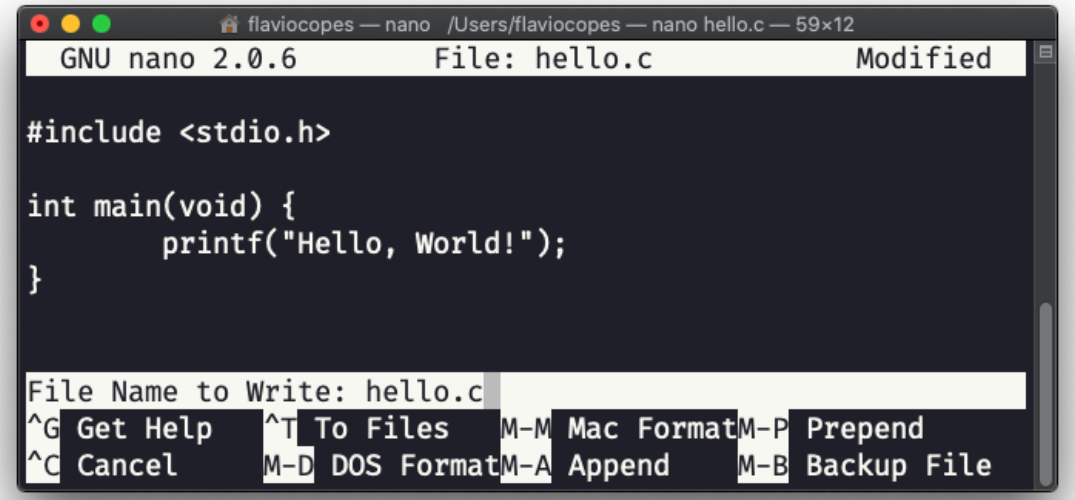

Figure 6: Screen-Shot-2020-01-29-at-10.18.15

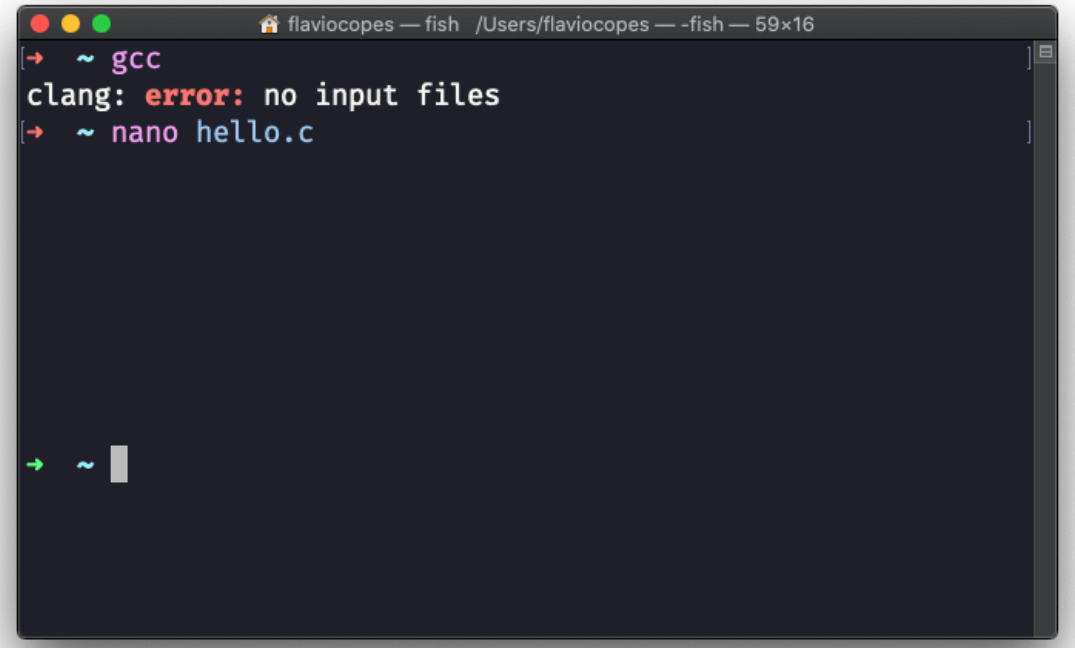

Figure 7: Screen-Shot-2020-01-29-at-10.13.46

```
gcc hello.c -o hello
```

O programa não deve gerar erros:

mas deve ter gerado um executável chamado `hello`. Agora, digite

```
./hello
```

para executar o programa:

Antes do nome do programa, eu adicionei `./` para informar ao terminal que o comando está na pasta atual.

Ótimo!

Agora, se você chamar `ls -al hello`, poderá ver que o programa tem apenas 12KB de tamanho:

Essa é uma das vantagens do C: ele é altamente otimizado, o que também é uma das razões para que ele seja bom para dispositivos integrados que tenham uma quantidade limitada de recursos.

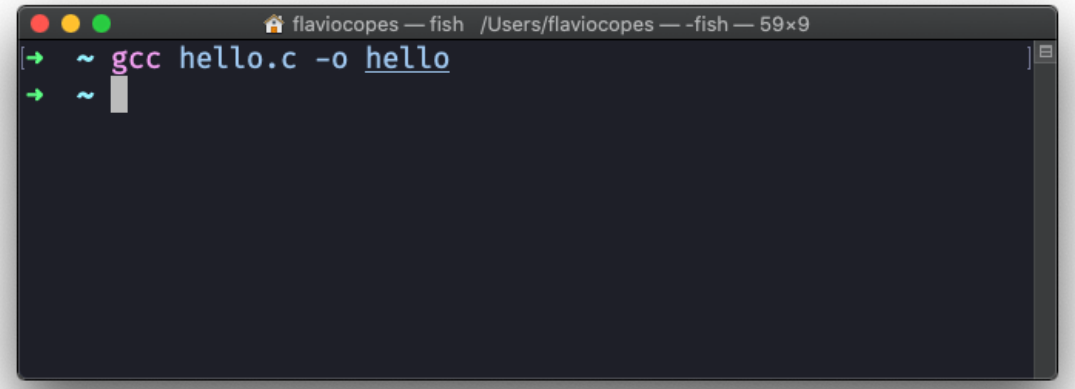

Figure 8: Screen-Shot-2020-01-29-at-10.16.31

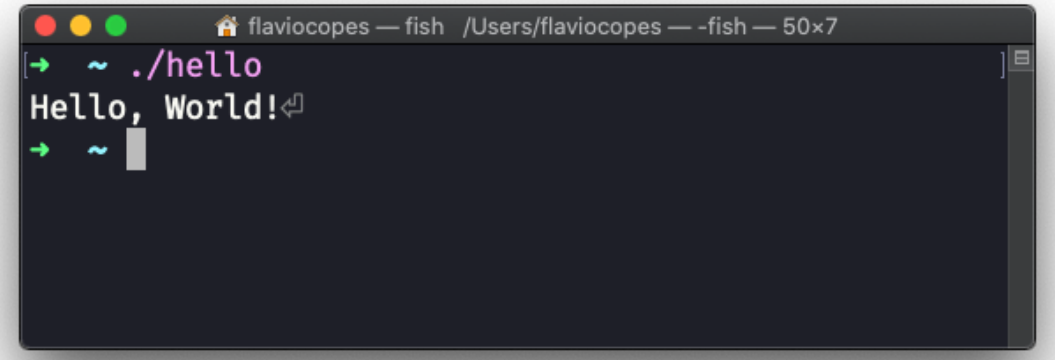

Figure 9: Screen-Shot-2020-01-29-at-10.19.20

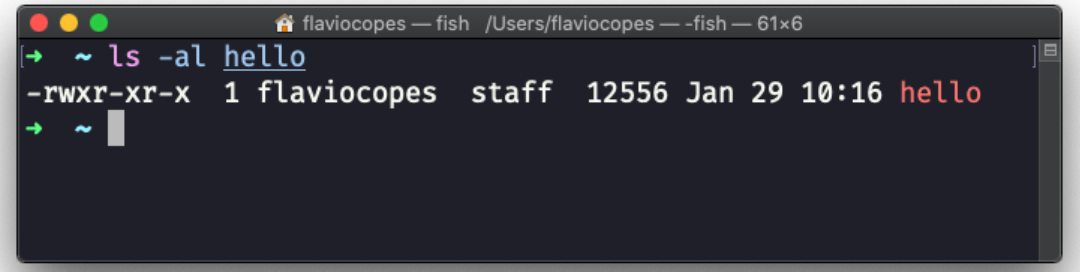

Figure 10: Screen-Shot-2020-01-29-at-10.19.55

Variáveis e tipos

Voltar ao sumário.

C é uma linguagem estaticamente tipada.

Isso significa que todas as variáveis têm um tipo associado. Esse tipo é conhecido em tempo de compilação.

É bem diferente da maneira como trabalhamos com variáveis em Python, JavaScript, PHP e outras linguagens interpretadas.

Ao criar uma variável em C, você tem de especificar o tipo de uma variável na declaração.

Neste exemplo, inicializamos a variável `age` com o tipo `int`:

```
int age;
```

O nome de uma variável pode conter letras maiúsculas e minúsculas, algarismo e o caractere de sublinha (*underscore*, em inglês), mas não podem começar por um algarismo. `AGE` e `Age10` são nomes válidos para variáveis, mas `1age` não é.

Você também pode iniciar uma variável no momento da declaração, especificando seu valor inicial:

```
int age = 37;
```

Ao declarar uma variável, você pode usá-la no código do programa. Você pode alterar seu valor a qualquer momento, usando o operador `=`, por exemplo, como em `age = 100;` (contanto que o novo valor seja do mesmo tipo).

Neste caso:

```
#include <stdio.h>

int main(void) {
    int age = 0;
    age = 37.2;
    printf("%u", age);
}
```

o compilador lançará um aviso no momento da compilação e converterá o número decimal em um valor inteiro.

Os tipos de dados integrados do C são `int`, `char`, `short`, `long`, `float`, `double` e `long double`. Vejamos um pouco mais a respeito deles.

Números inteiros

O C nos fornece os seguintes tipos para que possamos definir valores inteiros:

- `char`
- `int`
- `short`
- `long`

Na maior parte das vezes, você provavelmente usará um `int` para armazenar um número inteiro. Em alguns casos, no entanto, você pode querer escolher uma das outras três opções.

O tipo `char` normalmente é usado para armazenar letras da tabela ASCII, mas pode ser usada para conter números inteiros pequenos, entre `-128` e `127`. Ele ocupa menos de 1 byte.

`int` ocupa pelo menos 2 bytes. `short` ocupa pelo menos 2 bytes. `long` ocupa pelo menos 4 bytes.

Como você pode ver, não temos a garantia dos mesmos valores para ambientes diferentes. Temos apenas uma indicação. O problema é que os números exatos que podem ser armazenados em cada tipo de dados dependem da implementação e da arquitetura.

O que temos garantido é que `short` não é maior do que `int` e que `long` não é menor do que `int`.

O padrão das especificações ANSI C determina os valores mínimos para cada tipo. Graças a isso, podemos ao menos saber qual é o valor mínimo que podemos esperar ter à nossa disposição.

Se você estiver programando em C em um Arduino, placas de circuito diferentes terão limites diferentes.

Em uma placa Arduino Uno, `int` armazena um valor de 2 bytes, indo de `-32,768` a `32,767`. Em uma placa Arduino MKR 1010, `int` armazena um valor de 4 bytes, indo de `-2,147,483,648` a `2,147,483,647`, o que traz uma grande diferença.

Em todas as placas Arduino, `short` armazena um valor de 2 bytes, indo de `-32,768` a `32,767`. `long` armazena 4 bytes, indo de `-2,147,483,648` a `2,147,483,647`.

Números inteiros sem sinal

Para todos os tipos de dados acima, podemos anexar antes deles a expressão `unsigned` (em português, sem sinal) para iniciar o intervalo em 0, em vez de em um número negativo. Isso pode fazer sentido em diversos casos.

- `unsigned char` irá de `0` a pelo menos `255`
- `unsigned int` irá de `0` a pelo menos `65,535`
- `unsigned short` irá de `0` a pelo menos `65,535`
- `unsigned long` irá de `0` a pelo menos `4,294,967,295`

O problema com o overflow

Em função de todos esses limites, pode surgir uma pergunta: como podemos nos certificar de que nossos números não excedem o limite? O que acontece se excedermos o limite?

Se você tem um número `unsigned int` de 255 e somar mais um a ele, terá um retorno de 256, conforme o esperado. Se tiver um número `unsigned char` de 255 e somar mais um a ele, terá um retorno de 0. Ele retornará ao valor inicial possível.

Se você tiver um número `unsigned char` de 255 e adicionar 10 a ele, chegará ao número `9`:

```
#include <stdio.h>

int main(void) {
  unsigned char j = 255;
  j = j + 10;
  printf("%u", j); /* 9 */
}
```

Se você não tiver um valor com sinal, o comportamento é indefinido. Ele, basicamente, fornecerá um número enorme e que pode variar, como neste caso:

```
#include <stdio.h>

int main(void) {
  char j = 127;
  j = j + 10;
  printf("%u", j); /* 4294967177 */
}
```

Em outras palavras, o C não protege você de passar dos limites de um tipo. Você é quem deve se preocupar com essa questão.

Avisos ao declarar o tipo errado

Ao declarar a variável e inicializá-la com um valor do tipo incorreto, o compilador `gcc` (o que você provavelmente usará) deverá dar um aviso semelhante a este:

```c
#include <stdio.h>

int main(void) {
  char j = 1000;
}
```

```
hello.c:4:11: warning: implicit conversion
  from 'int' to
      'char' changes value from 1000 to -24
      [-Wconstant-conversion]
        char j = 1000;
             ~   ^~~~
1 warning generated.
```

Ele também avisará você no caso de atribuições diretas:

```c
#include <stdio.h>

int main(void) {
  char j;
  j = 1000;
}
```

Ele não fará isso, porém, se você aumentar o número usando, por exemplo, `+=`:

```c
#include <stdio.h>

int main(void) {
  char j = 0;
  j += 1000;
}
```

Números de ponto flutuante

Os tipos de ponto flutuante (`float`, `double` e `long double`) podem representar um conjunto muito maior de valores do que os inteiros, podendo, também, representar frações, algo que os números inteiros não conseguem fazer.

Ao usar números de ponto flutuante, representamos números como decimais na potência de 10.

Você poderá ver números de ponto flutuante escritos assim:

- `1.29e-3`

- `-2.3e+5`

além de outros escritos de maneira estranha.

Os tipos:

- `float`
- `double`
- `long double`

são usados para representar números com pontos decimais (tipos de ponto flutuante). Todos podem representar números positivos e negativos.

Os requisitos mínimos para qualquer implementação em C são de que `float` possa representar um intervalo entre 10^-37 e 10^+37, sendo tipicamente implementados usando 32 bits. `double` pode representar um conjunto mais de números. `long double` pode ter ainda mais números.

Os valores exatos, assim como ocorre com os números inteiros, dependem da implementação.

Em um Mac moderno, um `float` é representado por 32 bits, tendo uma precisão de 24 bits significativos. 8 bits são usados para codificar o expoente.

Um número `double` é representado por 64 bits, com uma precisão de 53 bits significativos, sendo usados 11 bits para codificar o expoente.

O tipo `long double` é representado por 80 bits, tem uma precisão de 64 bits significativos, sendo usados 15 bits para codificar o expoente.

Em nosso computador especificamente, como podemos determinar o tamanho dos tipos? Você pode escrever um programa que faça isso por você:

```c
#include <stdio.h>

int main(void) {
  printf("Tamanho de char: %lu bytes\n", sizeof(char));
  printf("Tamanho de int: %lu bytes\n", sizeof(int));
  printf("Tamanho de short: %lu bytes\n", sizeof(short));
  printf("Tamanho de long: %lu bytes\n", sizeof(long));
  printf("Tamanho de float: %lu bytes\n", sizeof(float));
  printf("Tamanho de double: %lu bytes\n",
    sizeof(double));
  printf("Tamanho de long double: %lu bytes\n",
    sizeof(long double));
}
```

Em meu sistema, um Mac moderno, veremos:

```
Tamanho de char: 1 bytes
Tamanho de int: 4 bytes
Tamanho de short: 2 bytes
Tamanho de long: 8 bytes
Tamanho de float: 4 bytes
Tamanho de double: 8 bytes
Tamanho de long double: 16 bytes
```

Constantes

Voltar ao sumário.

Vamos, agora, falar das constantes.

Uma constante é declarada de modo semelhante às variáveis, exceto pelo fato de ter a palavra-chave `const` adicionada antes da declaração. Além disso, você sempre precisará adicionar na declaração um valor específico.

Por exemplo:

```
const int age = 37;
```

Esse código é perfeitamente válido em C, mas é comum declarar as constantes em letra maiúscula, assim:

```
const int AGE = 37;
```

É apenas uma convenção, mas pode ajudar muito ao ler ou escrever um programa em C, já que aumenta a legibilidade. Um nome em letras maiúsculas significa que é uma constante, enquanto, em letras minúsculas, temos uma variável.

Um nome de constante segue as mesmas regras dos nomes de variáveis: podem conter letras maiúsculas e minúsculas, algarismos e o caractere de sublinha, mas não pode começar com um algarismo. `AGE` e `Age10` são nomes de variável válidos, enquanto `1AGE` não é.

Outra maneira de definir constantes é usando essa sintaxe:

```
#define AGE 37
```

Neste caso, não é necessário adicionar um tipo, também não sendo preciso adicionar um sinal de igual `=`. Você deve omitir o ponto e vírgula do final.

O compilador do C fará a inferência do tipo a partir do valor especificado, em tempo de compilação.

Operadores

Voltar ao sumário.

O C nos oferece uma grande variedade de operadores que podemos usar para fazer operações com os dados.

Especificamente, podemos identificar alguns grupos de operadores:

- operadores aritméticos
- operadores de comparação
- operadores lógicos
- operadores de atribuição compostos
- operadores bitwise
- operadores de ponteiro
- operadores de estrutura
- operadores diversos

Nesta seção, detalharei todos eles usando 2 variáveis imaginárias, `a` e `b`, como exemplo.

Deixarei os operadores bitwise, de estrutura e de ponteiro fora dessa lista para manter o guia simples

Operadores aritméticos

Neste grupo mais amplo, separarei os operadores binários e os unários.

Os operadores binários funcionam usando dois operandos:

OPERADOR	NOME	EXEMPLO
=	Atribuição	a = b
+	Adição	a + b
-	Subtração	a - b
*	Multiplicação	a * b
/	Divisão	a / b
%	Módulo	a % b

Operadores unários recebem apenas um operando:

OPERADOR	NOME	EXEMPLO
+	Unário de soma	+a
-	Unário de subtração	-a
++	Incremento	a++ or ++a

OPERADOR	NOME	EXEMPLO
–	Decremento	a– or –a

A diferença entre `a++` e `++a` é que `a++` incrementa a variável `a` após seu uso. `++a` incrementa a variável `a` antes de usá-la.

Por exemplo:

```
int a = 2;
int b;
b = a++ /* b é 2, a é 3 */
b = ++a /* b é 4, a é 4 */
```

Isso também se aplica aos operadores de decremento.

Operadores de comparação

OPERADOR	NOME	EXEMPLO
==	Operador de igual	a == b
!=	Operador de diferente	a != b
>	Maior que	a > b
<	Menor que	a < b
>=	Maior que ou igual a	a >= b
<=	Menor que ou igual a	a <= b

Operadores lógicos

- `!` NOT (em português, "não" - exemplo: `!a`)
- `&&` AND (em português, "e" - exemplo: `a && b`)
- `||` OR (em português, "ou" - exemplo: `a || b`)

Esses operadores são ótimos ao se trabalhar com valores booleanos.

Operadores de atribuição compostos

Esses operadores são úteis para realizar uma atribuição e, ao mesmo tempo, realizar uma operação aritmética:

OPERADOR	NOME	EXEMPLO
+=	Atribuição e adição	a += b
-=	Atribuição e subtração	a -= b

OPERADOR	NOME	EXEMPLO
*=	Atribuição e multiplicação	a *= b
/=	Atribuição e divisão	a /= b
%=	Atribuição e módulo	a %= b

O operador ternário

O operador ternário é o único operador em C que funciona com 3 operandos, sendo um modo abreviado de expressar condicionais.

Ele tem essa aparência:

```
<condição> ? <expressão> : <expressão>
```

Exemplo:

```
a ? b : c
```

Se `a` for avaliado como `true`, a instrução `b` será executada. Caso contrário, é executada a instrução `c`.

O operador ternário, em termos de sua funcionalidade, é equivalente ao condicional if/else, exceto pelo fato de ser menor e de podermos colocar nele toda uma expressão em uma única linha.

sizeof

O operador `sizeof` retorna o tamanho do operando que você passar. É possível passar uma variável ou, até mesmo, um tipo.

Exemplo de uso:

```c
#include <stdio.h>

int main(void) {
  int age = 37;
  printf("%ld\n", sizeof(age));
  printf("%ld", sizeof(int));
}
```

Precedência dos operadores

Com todos esses operadores (e mais, já que não tratamos de todos neste artigo, incluindo os operadores bitwise, de estrutura e de ponteiro), devemos prestar atenção a quando eles estiverem juntos em uma única expressão.

Vamos supor que temos esta operação:

```
int a = 2;
int b = 4;
int c = b + a * a / b - a;
```

Qual é o valor de `c`? A adição é executada antes da multiplicação e da divisão?

Existe um conjunto de regras que nos ajuda a resolver esse enigma.

Na ordem de menor precedência para a maior, temos:

- o operador de atribuição `=`
- os operadores **binários** `+` e `-`
- os operadores `*` e `/`
- os operadores `+` e `-` unários

Os operadores também têm uma regra de associação, que é sempre da esquerda para a direita, exceto para os operadores unários e de atribuição.

Em:

```
int c = b + a * a / b - a;
```

Primeiro, executamos `a * a / b`, que, devido ao fato de ser da esquerda para a direita, separamos em `a * a` e o resultado `/ b`: `2 * 2 = 4`, `4 / 4 = 1`.

Depois, realizamos a soma e a subtração: 4 + 1 - 2. O valor de `c` é `3`.

Em todos os casos, no entanto, eu gostaria de reforçar que é possível usar os parênteses de modo a tornar expressões semelhantes mais fáceis de ler e de compreender.

Os parênteses têm maior prioridade sobre todo o resto.

A expressão do exemplo acima pode ser reescrita assim:

```
int c = b + ((a * a) / b) - a;
```

e, desse modo, não temos que nos preocupar muito com ela.

Condicionais

Voltar ao sumário.

Todas as linguagens de programação fornecem aos programadores a capacidade de realizar escolhas.

Em alguns casos, queremos realizar a ação X. Em outros, a ação Y.

Queremos verificar os dados e fazer escolhas com base no estado desses dados.

O C nos fornece duas maneiras de fazer isso.

A primeira é a instrução `if`, com seu auxiliar `else`, e a segunda é a instrução `switch`.

if

Em uma instrução `if` (*se*, em português), você pode conferir se uma condição é verdadeira e executar o bloco fornecido entre chaves:

```
int a = 1;

if (a == 1) {
  /* Realizar uma ação */
}
```

Você pode anexar um bloco `else` (senão, em português) para executar um bloco diferente se a condição não for verdadeira (em outras palavras, se ela for falsa):

```
int a = 1;

if (a == 2) {
  /* Realizar uma ação */
} else {
  /* Realizar outra ação, caso contrário */
}
```

Cuidado com uma fonte comum de erros - sempre use o operador de comparação `==` para fazer comparações, não o operador de atribuição `=`. Do contrário, a verificação do condicional `if` será sempre verdadeira, a menos que o argumento seja `0`. Por exemplo, se você executar esse código:

```
int a = 0;

if (a = 0) {
```

```
  /* O que estiver aqui nunca será chamado */
}
```

Por que isso acontece? Porque a verificação do condicional buscará um resultado booleano (*true*/*false*, verdadeiro ou falso, o resultado de uma comparação). O número `0` sempre será igual ao valor *false*. Todo o resto será igual a *true*, incluindo números negativos.

Você também pode ter diversos blocos de `else` empilhando-os na forma de instruções `else if`:

```
int a = 1;

if (a == 2) {
  /* Realizar uma ação */
} else if (a == 1) {
  /* Realizar outra ação */
} else {
  /* Realizar uma terceira ação */
}
```

switch

Caso precise fazer muitos blocos *if*/*else*/*else if* para realizar uma verificação, talvez por precisar verificar o valor exato de uma variável, a instrução `switch` (algo como *chave* em português) pode ser bastante útil para você.

Você pode fornecer uma variável como condição e uma série de pontos de entrada `case` (*caso*, em português) para cada valor que você espera:

```
int a = 1;

switch (a) {
  case 0:
    /* Realizar uma ação */
    break;
  case 1:
    /* Realizar outra ação */
    break;
  case 2:
    /* Realizar outra ação */
    break;
}
```

Precisamos usar a palavra-chave `break` ao final de cada *case* para evitar que o próximo *case* seja executado depois de o anterior ter sido concluído. Esse efeito "cascata" pode ser útil de maneiras bastante criativas.

Você pode usar um *case* que valha para várias situações ao final, chamado de `default`:

```
int a = 1;

switch (a) {
  case 0:
    /* Realizar uma ação */
    break;
  case 1:
    /* Realizar outra ação */
    break;
  case 2:
    /* Realizar outra ação */
    break;
  default:
    /* Para todos os outros casos */
    break;
}
```

Laços

Voltar ao sumário.

O C nos oferece três maneiras de realizar um laço: **laços** for****, **laços w**hile**** e **laços do** while****. Todos eles permitem percorrer os elementos de um array, mas com algumas diferenças. Vamos vê-los em detalhes.

Laços for

O primeiro modo e, provavelmente, o mais comum de se realizar um laço é usando **laços f**or****.

Usando a palavra-chave `for`, podemos definir as ___r_egras_ para o laço desde o início e, então, fornecer o bloco que será executado repetidamente.

Aqui vemos um exemplo:

```
for (int i = 0; i <= 10; i++) {
  /* as instruções a serem repetidas */
}
```

O bloco `(int i = 0; i <= 10; i++)` contém 3 partes com os detalhes do laço:

- a condição inicial (`int i = 0`)
- o teste (`i <= 10`)
- o incremento (`i++`)

Definimos, primeiramente, a variável do laço. Neste caso, a chamamos de `i`. `i` é uma variável comum que será usada em laços, assim como `j` para laços aninhados (um laço dentro de outro laço). Ambas são usadas apenas por convenção.

A variável é inicializada com o valor de 0 (zero) e a primeira iteração é realizada. Logo, vem a parte do incremento (`i++`, neste caso, que incrementa a variável em 1), e todo o ciclo é repetido até você receber o número 10.

Dentro do bloco principal do laço, podemos acessar a variável `i` para saber em que iteração estamos. Este programa deve imprimir `0 1 2 3 4 5 5 6 7 8 9 10`:

```
for (int i = 0; i <= 10; i++) {
  /* as instruções a serem repetidas */
  printf("%u ", i);
}
```

Laços podem iniciar em um número maior e diminuir de acordo, assim:

```
for (int i = 10; i > 0; i--) {
```

```
  /* as instruções a serem repetidas */
}
```

Você também pode incrementar a variável de laço de 2 em 2 ou de acordo com algum outro valor:

```
for (int i = 0; i < 1000; i = i + 30) {
  /* as instruções a serem repetidas */
}
```

Laços while

Os laços while**** são mais simples de escrever do que os laços `for`, mas exige um pouco mais de trabalho de sua parte.

Em vez de definir todo o laço de início ao começar o laço, como é feito no laço `for`, usando `while`, apenas verificamos uma condição:

```
while (i < 10) {

}
```

Esta maneira de escrever leva em consideração que `i` já esteja definida e inicializada com um valor.

Esse laço, porém, será um **laço i**nfinit**o**, a menos que você incremente a variável `i` em algum momento, dentro do laço. Um laço infinito é ruim, pois bloqueará o programa, não permitindo que mais nada aconteça.

É isso que você precisa para fazer um laço while "correto":

```
int i = 0;

while (i < 10) {
  /* realizar uma ação */

  i++;
}
```

Existe uma exceção a isso - e a veremos em um minuto. Antes, permitam-me apresentar o laço `do while`.

Laços do while

Embora os laços while sejam ótimos, pode haver situações em que você precise fazer uma coisa específica: executar um bloco ao menos uma vez *sempre* e, *talvez*, repeti-lo.

Isso é feito usando as palavras-chave `do while`. De certa forma, ele é muito semelhante a um laço `while`, com uma única diferença:

```
int i = 0;

do {
  /* realizar uma ação */

  i++;
} while (i < 10);
```

O bloco que contém o comentário `/* realizar uma ação */` sempre é executado ao menos uma vez, não importando a verificação da condição abaixo.

Então, enquanto a variável `i` for inferior a 10, o bloco será repetido.

Saindo de um laço usando *break*

Em todos os laços da linguagem C, temos um modo de sair do laço a qualquer momento, imediatamente, não importando as condições definidas pelo laço.

Isso é feito usando a palavra-chave `break`.

Isso é útil em várias situações. Você pode querer verificar o valor de uma variável, por exemplo:

```
for (int i = 0; i <= 10; i++) {
  if (i == 4 && someVariable == 10) {
    break;
  }
}
```

Essa opção de sair de um laço é particularmente interessante para laços `while` (e `do while`), pois podemos criar laços aparentemente infinitos que se encerram quando uma condição ocorre. Você define isso dentro do bloco do laço:

```
int i = 0;
while (1) {
  /* realizar uma ação */

  i++;
  if (i == 10) break;
}
```

É muito comum termos esse tipo de laço em C.

Arrays

Voltar ao sumário.

Um array é uma variável que armazena diversos valores.

Todos os valores em um array, em C, precisam ter o **mesmo tipo**. Em outras palavras, teremos apenas arrays de valores `int`, arrays de valores `double` e assim por diante.

Você pode definir um array de valores `int`, assim:

```
int prices[5];
```

Você deve sempre especificar o tamanho do array. O C não fornece arrays dinâmicos nativamente (é preciso usar uma estrutura de dados, como uma lista vinculada, para que isso seja possível).

Você pode usar uma constante para definir o tamanho:

```
const int SIZE = 5;
int prices[SIZE];
```

Você pode inicializar um no momento da definição, assim:

```
int prices[5] = { 1, 2, 3, 4, 5 };
```

Além disso, você pode atribuir um valor após a definição, desta forma:

```
int prices[5];

prices[0] = 1;
prices[1] = 2;
prices[2] = 3;
prices[3] = 4;
prices[4] = 5;
```

Ou ainda, de um modo mais prático, usando um laço:

```
int prices[5];

for (int i = 0; i < 5; i++) {
  prices[i] = i + 1;
}
```

Você também pode referenciar um item no array usando colchetes após o nome da variável array, adicionando um número inteiro para determinar o valor do índice, assim:

```
prices[0]; /* valor do item do array: 1 */
prices[1]; /* valor do item do array: 2 */
```

Os índices do array iniciam em 0 (zero). Desse modo, um array com 5 itens, como o array `prices` acima, terá itens que vão de `prices[0]` até `prices[4]`.

Algo interessante sobre os arrays em C é o fato de que todos os elementos são armazenados em sequência, um após o outro. Isso não é algo que normalmente ocorra com linguagens de programação de alto nível.

Outra questão que vale a pena saber é sobre o nome da variável do array, como `prices`, no exemplo acima. O nome é um ****po**nteiro** do primeiro elemento do array. Assim, ele pode ser usado como um ponteiro normal.

Falaremos mais sobre ponteiros adiante.

Strings

Voltar ao sumário.

Em C, as strings são um tipo especial de array: uma string é um array de valores `char`:

```
char name[7];
```

Apresentei o tipo `char` quando falamos em tipos, mas, resumidamente, ele é comumente usado para armazenar letras da tabela ASCII.

Uma string pode ser inicializada da mesma maneira como você inicializa um array normal:

```
char name[7] = { "F", "l", "a", "v", "i", "o" };
```

Ou ainda, de modo mais conveniente, como uma *string literal* (também chamada de *string constante*), uma sequência de caracteres envoltas em aspas duplas:

```
char name[7] = "Flavio";
```

Você pode imprimir uma string com `printf()` usando `%s`:

```
printf("%s", name);
```

Percebeu como "Flavio" tem 6 caracteres, mas eu defini um array de tamanho 7? Por quê? Isso ocorre porque o último caractere de uma string deve ser um valor `0`, o finalizador da string e precisamos deixar um espaço para ele.

É importante ter isso em mente, especialmente ao manipular strings.

Falando em manipular strings, existe uma biblioteca padrão importante fornecida pelo C: `string.h`.

Essa biblioteca é essencial, pois ela abstrai muitos dos detalhes de baixo nível ao trabalhar com strings e nos fornece um conjunto de funções úteis.

Você pode carregar a biblioteca em seu programa, adicionando-o na parte superior:

```
#include <string.h>
```

Quando fizer isso, você terá acesso a:

- `strcpy()` para copiar uma string sobre outra
- `strcat()` para anexar uma string à outra
- `strcmp()` para comparar se duas strings são iguais
- `strncmp()` para comparar os primeiros `n` caracteres de duas strings
- `strlen()` para calcular o tamanho de uma string

e muito, muito mais.

Ponteiros

Voltar ao sumário.

Os ponteiros são uma das partes mais confusas/desafiadoras da linguagem C, na minha opinião. Em especial, se você está recém começando em programação, mas também se você vier de uma linguagem de programação de alto nível, como o Python ou o JavaScript.

Nesta seção, eu quero apresentá-los da maneira mais simples possível, mas sem imbecilizar a definição.

Um ponteiro é o endereço de um bloco de memória que contém uma variável.

Quando você declara um número inteiro assim:

```
int age = 37;
```

É possível usar o operador `&` para obter o valor do endereço na memória de uma variável:

```
printf("%p", &age); /* 0x7ffeef7dcb9c */
```

Usei o formato `%p` especificado em `printf()` para imprimir o valor do endereço.

Podemos atribuir o endereço a uma variável:

```
int *address = &age;
```

Usando `int *address` na declaração, não estamos declarando uma variável inteira, mas um ****po**nteiro para um número inteiro**.

Podemos usar o operador de ponteiro `*` para obter o valor da variável para a qual um endereço está apontando:

```
int age = 37;
int *address = &age;
printf("%u", *address); /* 37 */
```

Dessa vez, estamos usando o operador de ponteiro novamente, mas, como ele não é uma declaração agora, isso significa "o valor da variável para a qual esse ponteiro está apontando".

Neste exemplo, declaramos uma variável `age` e usamos um ponteiro para inicializar o valor:

```
int age;
int *address = &age;
*address = 37;
printf("%u", *address);
```

Ao trabalhar com o C, você verá que várias coisas são construídas tendo como base esse conceito simples. Certifique-se, então, de se familiarizar um pouco com ele executando os

exemplos acima por conta própria.

Ponteiros são uma grande oportunidade, pois eles nos forçam a pensar sobre endereços de memória e sobre como os dados são organizados.

Arrays são um exemplo. Quando você declara um array:

```
int prices[3] = { 5, 4, 3 };
```

A variável `prices` é, de fato, um ponteiro para o primeiro item do array. Você pode obter o valor do primeiro item usando essa função `printf()`, neste caso:

```
printf("%u", *prices); /* 5 */
```

O interessante aqui é que podemos obter o segundo item adicionando 1 ao ponteiro de `prices`:

```
printf("%u", *(prices + 1)); /* 4 */
```

E assim por diante, para todos os outros valores.

Também podemos fazer muitas operações de manipulação de strings interessantes, já que strings são arrays internamente.

Também temos muitas outras aplicações, incluindo passar a referência de um objeto ou de uma função para evitar de consumir mais recursos ao copiar esse objeto ou essa função.

Funções

Voltar ao sumário.

As funções são uma forma através da qual podemos estruturar nosso código em sub-rotinas as quais podemos:

1. dar um nome
2. chamar quando precisamos delas

Iniciando de seu primeiro programa, um "Hello, World!", você já faz uso de funções em C imediatamente:

```
#include <stdio.h>

int main(void) {
    printf("Hello, World!");
}
```

A função `main()` é muito importante, pois é o ponto de entrada para um programa em C.

Aqui temos outra função:

```
void fazerAlgo(int value) {
    printf("%u", value);
}
```

As funções têm quatro aspectos importantes:

1. elas têm um nome, então podemos invocá-las ("chamá-las") mais tarde
2. elas especificam um valor de retorno
3. elas podem ter argumentos
4. elas têm um corpo, envolvido por chaves

O corpo da função é o conjunto de instruções que são executadas sempre que invocamos uma função.

Se a função não tiver um valor de retorno, você pode usar a palavra-chave `void` antes do nome da função. Do contrário, especifique o tipo do valor de retorno da função (`int` para inteiros, `float` para valores de ponto flutuante, `const char *` para uma string etc.).

Você não pode retornar mais de um valor a partir de uma função.

Uma função pode ter argumentos. Eles são opcionais. Se ela não os tiver, dentro dos parênteses, inserimos `void`, assim:

```
void fazerAlgo(void) {
   /* ... */
```

```
}
```

Neste caso, quando invocamos a função, a chamamos sem nada entre os parênteses:

```
fazerAlgo();
```

Se nós temos um parâmetro, especificamos o tipo e o nome do parâmetro, assim:

```
void fazerAlgo(int valor) {
   /* ... */
}
```

Quando invocamos o nome da função, passamos aquele parâmetro nos parênteses, assim:

```
fazerAlgo(3);
```

Podemos ter vários parâmetros, e, se tivermos, os separamos usando uma vírgula, na declaração e na invocação:

```
void fazerAlgo(int valor1, int valor2) {
   /* ... */
}

fazerAlgo(3, 4);
```

Parâmetros são passados por ****c**ópia**. Isso significa que, se você modificar `valor1`, seu valor é modificado localmente. O valor fora da função, onde ele foi passado na invocação, não se altera.

Se você passar um ****po**nteiro** como parâmetro, pode modificar aquele valor de variável, pois agora você pode acessá-la diretamente, usando seu endereço de memória.

Você não pode definir um valor padrão como parâmetro. O C++ permite isso (assim como os programas na linguagem Arduino), mas o C não.

Não se esqueça de definir a função antes de chamá-la. Do contrário, o compilador lançará um aviso e um erro:

```
> ~ gcc hello.c -o hello; ./hello
hello.c:13:3: warning: implicit declaration of
      function 'fazerAlgo' is invalid in C99
      [-Wimplicit-function-declaration]
  fazerAlgo(3, 4);
  ^
hello.c:17:6: error: conflicting types for
      'fazerAlgo'
void fazerAlgo(int valor1, char valor2) {
```

```
    ^
hello.c:13:3: note: previous implicit declaration
      is here
  fazerAlgo(3, 4);
  ^
1 warning and 1 error generated.
```

O aviso tem a ver com a ordenação, algo que já mencionei antes.

O erro tem a ver com outra coisa que está relacionada. Como o C não "vê" a declaração da função antes da invocação, ele deve fazer suposições. Ele, então, supõe que a função retorne `int`. A função, no entanto, retorna `void`, que é a razão para o erro.

Se alterarmos a definição da função para:

```
int fazerAlgo(int valor1, int valor2) {
  printf("%d %d\n", valor1, valor2);
  return 1;
}
```

receberemos apenas o aviso, não o erro:

```
> ~ gcc hello.c -o hello; ./hello
hello.c:14:3: warning: implicit declaration of
      function 'fazerAlgo' is invalid in C99
      [-Wimplicit-function-declaration]
  fazerAlgo(3, 4);
  ^
1 warning generated.
```

De qualquer modo, certifique-se de declarar a função antes de usá-la. Mova a função para cima ou adicione o protótipo da função em um arquivo de cabeçalho (em inglês, *header*).

Dentro de uma função, é possível declarar variáveis.

```
void fazerAlgo(int valor) {
  int valorEmDobro = valor * 2;
}
```

Uma variável é criada no ponto de invocação da função e destruída quando a função se encerra. Ela não é visível de fora.

Dentro de uma função, você pode chamar a própria função. Isso se chama ****recurs**ão** e é algo que oferece possibilidades bem peculiares.

Entrada e saída

Voltar ao sumário.

O C é uma linguagem pequena, e o "núcleo" do C não inclui funcionalidades de entrada/saída (input/output, ou I/O).

Isso não é exclusivo do C, naturalmente. É comum o núcleo de uma linguagem não depender de I/O.

No caso do C, a entrada/saída é fornecida para nós por meio da biblioteca padrão (do inglês, *standard library)* do C por meio de um conjunto de funções definidas no arquivo de cabeçalho `stdio.h`.

Você pode importar essa biblioteca usando

```
#include <stdio.h>
```

na parte superior de seu arquivo em C.

Essa biblioteca nos fornece, entre muitas outras funções:

- `printf()`
- `scanf()`
- `sscanf()`
- `fgets()`
- `fprintf()`

Antes de descrever o que essas funções fazem, eu gostaria de dedicar uns instantes a falar dos ****streams** de I/O**.

Temos três tipos de streams de I/O no C:

- `stdin` (*standard input*, ou a entrada padrão)
- `stdout` (*standard output*, ou a saída padrão)
- `stderr` (*standard error*, ou o erro padrão)

Com funções de I/O, sempre trabalhamos com streams. Um stream é uma interface de alto nível que pode representar um dispositivo ou um arquivo. DO ponto de vista do C, não faz diferença ler de um arquivo ou ler de uma linha de comando: é um stream de I/O de todo modo.

Há algo, porém, que se deve ter em mente.

Algumas funções são projetadas para funcionar com um stream específico, como `printf()`, que usamos para imprimir caracteres no `stdout`. Usando seu equivalente mais geral, `fprintf()`, podemos especificar em qual stream queremos escrever.

Como eu comecei falando de `printf()`, vamos apresentá-lo agora.

`printf()` é uma das primeiras funções que você usará ao aprender programação em C.

Em sua forma mais simples, você passará a ela uma string literal:

```
printf("Olá!");
```

e o programa imprimirá o conteúdo da string na tela.

Você pode imprimir o valor de uma variável. Porém, isso é um pouco mais complicado, pois você precisa adicionar um caractere especial, um *placeholder*, que muda dependendo do tipo de variável. Por exemplo, usamos `%d` para um algarismo inteiro decimal com sinal:

```
int idade = 37;

printf("Tenho %d anos", idade);
```

Podemos imprimir mais de uma variável usando vírgulas:

```
int idade_ontem = 36;
int idade_hoje = 37;

printf("Ontem, eu tinha %d anos e hoje eu tenho %d", idade_ontem, idade_hoje);
```

Existem outros especificadores de formato, como o `%d`:

- `%c` para um caractere
- `%s` para um caractere
- `%f` para números de ponto flutuante
- `%p` para ponteiros

e muito mais.

Podemos usar caracteres de escape em `printf()`, como `\n`, usado para fazer com que a saída adicione uma nova linha.

`scanf()`

`printf()` é usada como uma função de saída. Quero apresentar uma função de entrada agora, para podermos dizer que vimos tanto a entrada quanto a saída: `scanf()`.

Essa função é usada para obter de um usuário que estiver executando o programa um valor, a partir da linha de comando.

Primeiro, devemos definir uma variável que conterá o valor que obtemos da entrada:

```
int idade;
```

Depois, chamamos `scanf()` com 2 argumentos: o formato (tipo) da variável e o endereço da variável:

```
scanf("%d", &idade);
```

Se quisermos obter uma string como entrada, devemos lembrar que o nome de uma string é um ponteiro para seu primeiro caractere. Assim, não é necessário o caractere `&` antes dela:

```
char nome[20];
scanf("%s", nome);
```

Aqui, vemos um pequeno programa que usa `printf()` e `scanf()`:

```
#include <stdio.h>

int main(void) {
  char nome[20];
  printf("Digite seu nome: ");
  scanf("%s", nome);
  printf("Foi digitado o nome %s", nome);
}
```

Escopo das variáveis

Voltar ao sumário.

Ao definir uma variável em um programa em C, dependendo de onde você a declarar, ela terá um **escopo** diferente.

Isso significa que ela estará disponível em alguns lugares, mas não em outros.

A posição determina 2 tipos de variáveis:

- **variáveis globais****
- **variáveis locais****

Esta é a diferença: uma variável declarada dentro de uma função é uma variável local, assim:

```
int main(void) {
  int idade = 37;
}
```

Variáveis locais estão acessíveis apenas dentro da função. Quando a função termina, elas deixam de existir. Elas são removidas da memória (com algumas exceções).

Uma variável definida fora de uma função é uma variável global, como neste exemplo:

```
int idade = 37;

int main(void) {
  /* ... */
}
```

As variáveis globais estarão acessíveis em qualquer função do programa, estando disponíveis durante toda a execução do programa, até que ele se encerre.

Já mencionei aqui que as variáveis locais não estão mais disponíveis depois do fim da função.

A razão é o fato de as variáveis locais serem declaradas na ****stack**** (em português, pilha) por padrão, a menos que você as aloque explicitamente na **heap** usando ponteiros. Nesse caso, você precisará gerenciar a memória por conta própria.

Variáveis estáticas

Voltar ao sumário.

Dentro de uma função, você pode inicializar uma **variável estática** usando a palavra-chave `static`.

Eu disse "dentro de uma função", pois as variáveis globais são estáticas por padrão, não havendo a necessidade de adicionar a palavra-chave.

O que é uma variável estática? Uma variável estática é inicializada se nenhum valor inicial for especificado, e retendo o valor em todas as chamadas da função.

Considere a função a seguir:

```
int aumentarIdade() {
  int idade = 0;
  idade++;
  return idade;
}
```

Se chamarmos `aumentarIdade()` uma vez, teremos `1` como valor de retorno. Se a chamarmos mais de uma vez, receberemos sempre 1 como retorno, pois `idade` é uma variável local e reinicializada como `0` em cada chamada de função.

Se alterarmos a função para:

```
int aumentarIdade() {
  static int idade = 0;
  idade++;
  return idade;
}
```

Agora, toda vez que chamarmos essa função, teremos um valor aumentado em um:

```
printf("%d\n", aumentarIdade());
printf("%d\n", aumentarIdade());
printf("%d\n", aumentarIdade());
```

retornará

```
1
2
3
```

Também podemos omitir a inicialização de `idade` como 0 em `static int idade = 0;` e simplesmente escrever `static int idade;`, pois variáveis estáticas são definidas auto-

maticamente como 0 ao serem criadas.

Também podemos ter arrays estáticos. Nesse caso, cada item do array é inicializado com 0:

```
int aumentarIdade() {
  static int idades[3];
  idades[0]++;
  return idades[0];
}
```

Variáveis globais

Voltar ao sumário.

Nesta seção, eu gostaria de falar um pouco mais sobre a diferença entre as **variáveis locais e globais**.

Uma **variável l**ocal**** é definida dentro de uma função, estando disponível apenas dentro daquela função.

Assim:

```
#include <stdio.h>

int main(void) {
  char j = 0;
  j += 10;
  printf("%u", j); //10
}
```

`j` não está disponível em nenhum outro lugar fora da função `main`.

Uma **variável g**lobal**** é definida fora das funções, assim:

```
#include <stdio.h>

char i = 0;

int main(void) {
  i += 10;
  printf("%u", i); //10
}
```

Uma variável global pode ser acessada de qualquer função do programa. O acesso não é limitado à leitura do valor: a variável pode ser atualizada a partir de qualquer função.

Por causa disso, variáveis globais são uma maneira que temos de compartilhar os mesmos dados entre funções.

A diferença principal delas para as variáveis locais é que a memória alocada para as variáveis é liberada assim que a função se encerra.

Variáveis globais são liberadas apenas quando o programa se encerra.

Definições de tipo

Voltar ao sumário.

A palavra-chave `typedef` em C permite que você defina tipos novos.

Partindo dos tipos integrados em C, podemos criar nossos próprios tipos, usando a sintaxe abaixo:

```
typedef tipoExistente NOVOTIPO
```

O novo tipo que criamos, em geral, estará em maiúsculas.

Isso serve para distingui-lo mais facilmente e para que o reconheçamos imediatamente como um tipo.

Por exemplo, podemos definir um novo tipo `NUMBER` como um `int`:

```
typedef int NUMBER
```

e, ao fazer isso, podemos definir novas variáveis `NUMBER`:

```
NUMBER um = 1;
```

Você pode estar se perguntando: para quê? Por que não usar simplesmente o tipo integrado `int` em vez disso?

Bem, `typedef` passa a ser realmente útil quando associado a outros dois conceitos: tipos enumerados e estruturas.

Tipos enumerados

Voltar ao sumário.

Usando as palavras-chave `typedef` e `enum`, podemos definir um tipo que pode ter um valor ou outro.

Esse é um dos usos mais importantes da palavra-chave `typedef`.

Esta é a sintaxe de um tipo enumerado:

```
typedef enum {
  //...valores
} NOME_DO_TIPO;
```

O tipo enumerado que criamos, em geral, por convenção, é em letras maiúsculas.

Aqui temos um exemplo simples:

```
typedef enum {
  true,
  false
} BOOLEANO;
```

C já vem com um tipo `bool`, o que faz com que esse exemplo não seja muito prático, mas você entende a ideia.

Outro exemplo é para a definição de dias da semana:

```
typedef enum {
  segunda,
  terca,
  quarta,
  quinta,
  sexta,
  sabado,
  domingo
} DIA_DA_SEMANA;
```

Aqui temos um programa simples que usa o tipo enumerado acima:

```
#include <stdio.h>

typedef enum {
  segunda,
  terca,
  quarta,
```

```
  quinta,
  sexta,
  sabado,
  domingo
} DIA_DA_SEMANA;

int main(void) {
  DIA_DA_SEMANA dia = segunda;

  if (dia == segunda) {
    printf("Hoje é segunda!");
  } else {
    printf("Hoje não é segunda");
  }
}
```

Cada item na definição de `enum` é pareado a um número inteiro internamente. Assim, nesse exemplo `segunda` é 0, `terca` é 1 e assim por diante.

Isso significa que o condicional poderia ter sido `if (dia == 0)` em vez de `if (dia == segunda)`, mas é muito mais simples para nós, seres humanos, entender os nomes do que os números, o que torna essa sintaxe mais conveniente.

Estruturas

Voltar ao sumário.

Usando a palavra-chave `struct`, podemos criar estruturas de dados complexas por meio dos tipos básicos em C.

Uma estrutura é uma coleção de valores de diferentes tipos. Arrays em C são limitados a um tipo, de modo que as estruturas podem se tornar bastante interessantes em muitos casos de uso.

Aqui vemos a sintaxe de uma estrutura:

```
struct <nome_da_estrutura> {
  //...variáveis
};
```

Exemplo:

```
struct pessoa {
  int idade;
  char *nome;
};
```

Você pode declarar variáveis que tenham como tipo aquela estrutura adicionando-as após a chave de fechamento, antes do ponto e vírgula, assim:

```
struct pessoa {
  int idade;
  char *nome;
} flavio;
```

Ou diversas variáveis, assim:

```
struct pessoa {
  int idade;
  char *nome;
} flavio, pessoas[20];
```

Nesse caso, eu declaro uma única variável `pessoa`, chamada `flavio`, e um array com 20 variáveis do tipo `pessoa`, chamado `pessoas`.

Também podemos declarar variáveis mais tarde, usando essa sintaxe:

```
struct pessoa {
  int idade;
  char *nome;
```

```
};

struct pessoa flavio;
```

Podemos inicializar uma estrutura no momento da declaração:

```
struct pessoa {
  int idade;
  char *nome;
};

struct pessoa flavio = { 37, "Flavio" };
```

e quando tivermos uma estrutura definida, podemos acessar os valores nela usando um ponto:

```
struct pessoa {
  int idade;
  char *nome;
};

struct pessoa flavio = { 37, "Flavio" };
printf("%s, idade %u", flavio.nome, flavio.idade);
```

Também podemos alterar os valores usando a sintaxe do ponto:

```
struct pessoa {
  int idade;
  char *nome;
};

struct pessoa flavio = { 37, "Flavio" };

flavio.idade = 38;
```

As estruturas são muito úteis, pois podemos passá-las como parâmetros de funções ou retornar valores, incorporando diversas variáveis dentro delas. Cada variável tem um rótulo.

É importante observar que as estruturas são ****pass**adas por cópia**, a menos que, logicamente, passemos um ponteiro a uma estrutura, caso em que ela é passada por referência.

Usando `typedef`, podemos simplificar o código ao trabalhar com estruturas.

Vejamos um exemplo:

```
typedef struct {
  int idade;
  char *nome;
} PESSOA;
```

A estrutura que criamos usando `typedef`, em geral, por convenção, fica em maiúsculas.

Agora, podemos declarar novas variáveis `PESSOA` assim:

```
PESSOA flavio;
```

e podemos inicializá-las no momento da declaração, assim:

```
PESSOA flavio = { 37, "Flavio" };
```

Parâmetros da linha de comando

Voltar ao sumário.

Em seus programas em C, você pode precisar aceitar parâmetros a partir da linha de comando quando um comando for executado.

Para necessidades mais simples, tudo o que você precisa fazer é alterar a assinatura da função `main()` de

```
int main(void)
```

para

```
int main (int argc, char *argv[])
```

`argc` é um número inteiro que contém o número de parâmetros que serão fornecidos na linha de comando.

`argv` é um array de strings.

Quando o programa é iniciado, recebemos os argumentos nesses 2 parâmetros.

Observe que sempre há ao menos um item no array `argv`: o nome do programa

Vamos pegar o exemplo do compilador do C que usamos para executar nossos programas, assim:

```
gcc hello.c -o hello
```

Se esse fosse o nosso programa, teríamos `argc` com o valor de 4 e `argv` sendo um array contendo

- `gcc`
- `hello.c`
- `-o`
- `hello`

Vamos escrever um programa que imprima os argumentos que ele recebe:

```
#include <stdio.h>

int main (int argc, char *argv[]) {
  for (int i = 0; i < argc; i++) {
    printf("%s\n", argv[i]);
  }
}
```

Se o nome do nosso programa for `hello` e se o executarmos assim: `./hello`, teremos, como resultado:

```
./hello
```

Se passarmos alguns parâmetros aleatórios, assim: `./hello a b c`, teremos o seguinte resultado no terminal:

```
./hello
a
b
c
```

Esse sistema funciona muito bem para necessidades simples. Para as mais complexas, existem pacotes usados normalmente, como o ****getopt****.

Arquivos de cabeçalho (headers)

Voltar ao sumário.

Programas simples podem ser colocados em um único arquivo. Porém, quando seu programa fica muito grande, é impossível mantê-lo inteiramente em um arquivo.

Você pode mover partes de um programa para um arquivo em separado. Você, então, cria um **arquivo de cabeçalho**.

Um arquivo de cabeçalho se parece com um arquivo normal em C, exceto pelo fato de terminar com um `.h` em vez de um `.c`. No lugar das implementações de suas funções e de outras partes de um programa, ele mantém as ****declara**ções**s****.

Você já usou arquivos de cabeçalho quando usou pela primeira vez a função `printf()` ou qualquer outra função de entrada/saída e precisou digitar:

```
#include <stdio.h>
```

para usá-la.

`#include` é uma diretiva do pré-processador.

O pré-processador procura pelo arquivo `stdio.h` na biblioteca padrão, pois você utilizado os sinais de `<` e `>` ao redor dele. Para incluir seus próprios arquivos de cabeçalho, use aspas, assim:

```
#include "meuArquivo.h"
```

A linha acima procurará por `meuArquivo.h` na pasta atual.

Você também pode usar uma estrutura de pastas para as bibliotecas:

```
#include "minhaPasta/meuArquivo.h"
```

Vejamos um exemplo. O programa a seguir calcula os anos que se passaram desde um determinado ano:

```
#include <stdio.h>

int calcularIdade(int ano) {
  const int ANO_ATUAL = 2022;
  return ANO_ATUAL - ano;
}

int main(void) {
  printf("%u", calcularIdade(1983));
}
```

Vamos supor que eu queira mover a função `calcularIdade` para um arquivo em separado.

Criamos, então, um arquivo `calcular_idade.c`:

```
int calcularIdade(int ano) {
  const int ANO_ATUAL = 2022;
  return ANO_ATUAL - ano;
}
```

E um arquivo `calcular_idade.h`, onde colocamos o *protótipo da f_un_ção*, que é o mesmo que a função no arquivo `.c`, exceto pelo corpo;

```
int calcularIdade(int ano);
```

Agora, no arquivo `.c` principal, podemos remover a definição da função `calcularIdade()` e importar `calcular_idade.h`, o que tornará disponível a função `calcularIdade()`:

```
#include <stdio.h>
#include "calcular_idade.h"

int main(void) {
  printf("%u", calcularIdade(1983));
}
```

Não se esqueça de que, para compilar um programa composto por diversos arquivos, é preciso listar todos eles na linha de comando, assim:

```
gcc -o main main.c calculate_age.c
```

Com configurações mais complexas, um Makefile é necessário para informar ao compilador como compilar o programa.

O pré-processador

Voltar ao sumário.

O pré-processador é uma ferramenta que nos ajuda muito quando programamos em C. Ele é parte do padrão C, assim como a linguagem, o compilador e a biblioteca padrão.

Ele analisa o nosso programa e garante que o compilador obtém tudo de que precisa antes de dar sequência ao processo.

O que ele faz, na prática?

Por exemplo, ele procura todos os arquivos de cabeçalho que você inclui com a diretiva `#include`.

Ele também procura todas as constantes que você definiu usando `#define` e as substitui pelo valor real.

Esse é só o começo. Eu mencionei essas 2 operações porque elas são as mais comuns. O pré-processador pode fazer isso e muito mais.

Você percebeu que `#include` e `#define` têm um `#` no começo? Isso é comum para todas as diretivas do pré-processador. Se uma linha começa com `#`, ela é tratada pelo pré-processador.

Condicionais de compilação

Voltar ao sumário.

Algo que podemos fazer é usar condicionais para mudar como nosso programa será compilado, dependendo do valor de uma expressão.

Por exemplo, podemos verificar se a constante `DEBUG` é 0:

```
#include <stdio.h>

const int DEBUG = 0;

int main(void) {
#if DEBUG == 0
  printf("Eu NÃO estou fazendo debugging\n");
#else
  printf("Eu estou fazendo debugging\n");
#endif
}
```

Constantes simbólicas

Podemos definir uma **constante simbólica**:

```
#define VALOR 1
#define PI 3.14
#define NOME "Flavio"
```

Quando usamos NOME, PI ou VALOR em nosso programa, o pré-processador substitui o nome das constantes pelo valor antes de executar o programa.

As constantes simbólicas são muito úteis, pois podemos dar nomes aos valores sem criar variáveis em tempo de compilação.

Macros

Com `#define`, também podemos definir uma ****macro****. A diferença entre uma macro e uma constante simbólica está no fato de que uma macro pode aceitar um argumento e tipicamente contém código, enquanto uma constante simbólica é um valor:

```
#define POTENCIA(x) ((x) * (x))
```

Observe os parênteses que cercam os argumentos: essa é uma prática recomendada para evitar problemas quando a macro é substituída no processo de pré-compilação.

Então, podemos usá-la em nosso código, assim:

```
printf("%u\n", POTENCIA(4)); //16
```

A grande diferença das funções está no fato de que as macros não especificam o tipo de seus argumentos ou valores de retorno, o que pode ser útil em alguns casos.

As macros, no entanto, estão limitadas a definições de uma linha.

If definido

Podemos verificar se uma constante simbólica ou uma macro estão definidas usando `#ifdef`:

```
#include <stdio.h>
#define VALOR 1

int main(void) {
#ifdef VALOR
  printf("O valor está definido\n");
#else
  printf("O valor não está definido\n");
#endif
}
```

Também temos `#ifndev` para verificar o contrário (se a macro não está definida).

Também podemos usar `#if defined` e `#if !defined` para realizar a mesma tarefa.

É comum envolver algum bloco de código em um bloco desse modo:

```
#if 0

#endif
```

para evitar temporariamente que ele seja executado, ou para usar uma constante simbólica DEBUG:

```
#define DEBUG 0

#if DEBUG
  //código enviado ao compilador
  //apenas se DEBUG não for 0
#endif
```

Constantes simbólicas predefinidas que você pode usar

O pré-processador também define algumas constantes simbólicas que você pode usar, identificadas por duas sublinhas antes e após o nome, incluindo:

- `__LINE__` representa a linha atual no arquivo do código-fonte
- `__FILE__` representa o nome do arquivo
- `__DATE__` representa a data de compilação, no formato `Mmm dd aaaa`
- `__TIME__` representa o tempo de compilação, no formato `hh:mm:ss`

Fonte: freecodecamp.